

The finite difference method

MATMEK-4270

Prof. Mikael Mortensen, University of Oslo

Initial or Boundary Value Problems

The second-order differential equation

$$u''(t) + au'(t) + bu(t) = f(t), \quad t \in [0, T]$$

is classified as either an IVP or a BVP based on boundary/initial conditions.

IVP

- Typically specifies $u(0)$ and $u'(0)$
- **Explicit** recurrence relations, no use of linear algebra
- Can use the finite difference or similar methods for discretization

BVP

- Typically specifies $u(0)$ and $u(T)$ or $u'(0)$ and $u'(T)$
- **Implicit** linear algebra methods
- Can use the finite difference or similar methods for discretization

Similar characteristics for higher-order differential equations. IVP specifies all conditions at initial time.

The explicit IVP approach simply computes u^{n+1} from $u^n, u^{n-1} \dots$

For example, the exponential decay problem with initial condition

$$u' + au = 0, t \in (0, T], u(0) = I.$$

is discretized as

$$u^{n+1} = \frac{1 - (1 - \theta)a\Delta t}{1 + \theta a\Delta t} u^n = gu^n.$$

The solution vector $\mathbf{u} = (u^0, u^1, \dots, u^N)$ is obtained from a recurrence algorithm

- $u^0 = I$
- for $n = 0, 1, \dots, N-1$
 - Compute $u^{n+1} = gu^n$

Easy to understand and easy to solve. Equations are never **assembled** into matrix form.
But may be unstable!

The BVP approach solves the difference equations by assembling matrices and vectors

Each **row** in the matrix-problem represents one difference equation.

$$Au = b$$

For the exponential decay problem the matrix problem looks like

$$\underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ -g & 1 & 0 & 0 & 0 \\ 0 & -g & 1 & 0 & 0 \\ 0 & 0 & -g & 1 & 0 \\ 0 & 0 & 0 & -g & 1 \end{bmatrix}}_A \underbrace{\begin{bmatrix} u^0 \\ u^1 \\ u^2 \\ u^3 \\ u^4 \end{bmatrix}}_u = \underbrace{\begin{bmatrix} I \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}}_b$$

Understand that the matrix problem is the same as the recurrence

$$\underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ -g & 1 & 0 & 0 & 0 \\ 0 & -g & 1 & 0 & 0 \\ 0 & 0 & -g & 1 & 0 \\ 0 & 0 & 0 & -g & 1 \end{bmatrix}}_A \underbrace{\begin{bmatrix} u^0 \\ u^1 \\ u^2 \\ u^3 \\ u^4 \end{bmatrix}}_u = \underbrace{\begin{bmatrix} I \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}}_b$$

simply means

$$\begin{aligned} u^0 &= I \\ -gu^0 + u^1 &= 0 \\ -gu^1 + u^2 &= 0 \\ &\vdots \end{aligned}$$

Still **explicit** since matrix is lower triangular. But problem is **assembled** into matrix form.

Solve matrix problem

The matrix problem

$$A\mathbf{u} = \mathbf{b}$$

is solved as

$$\mathbf{u} = A^{-1}\mathbf{b}$$

There are many different ways to achieve this. We will not be very concerned with how in this course.

Assemble matrix problem

```
1 a = 2
2 theta = 0.5
3 dt = 0.5
4 I = 1
5 N = 8
6 g = (1 - (1-theta) * a * dt)/(1 + theta * a * dt)
```

Assemble

```
1 from scipy import sparse
2 A = sparse.diags([-g, 1], np.array([-1, 0]), (N+1, N+1), 'csr')
3 b = np.zeros(N+1); b[0] = I
4 u = np.zeros(N+1)
```

Solve and compare with recurrence

Recurrence:

```
1 u[0] = I
2 for n in range(N):
3     u[n+1] = g * u[n]
```

Matrix

```
1 um = sparse.linalg.spsolve(A, b)
2 print(u-um)
```

```
[0. 0. 0. 0. 0. 0. 0. 0. 0.]
```

Ok, no difference. There's really no advantage of assembling the explicit exponential decay problem.

But which is faster?

```
1 def reccsolve(u, N, I):
2     u[0] = I
3     for n in range(N):
4         u[n+1] = g * u[n]
5 u = np.zeros(N+1)
6 %timeit -n100 reccsolve(u, N, I)
```

882 ns ± 55.7 ns per loop (mean ± std. dev. of 7 runs, 100 loops each)

```
1 %timeit -n100 um = sparse.linalg.spsolve(A, b)
```

26.3 µs ± 12.4 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)

Try to speed up by computing the inverse

A unit lower triangular matrix has a unit lower triangular inverse

```
1 np.set_printoptions(precision=3, suppress=True)
2 Ai = np.linalg.inv(A.toarray())
3 print(Ai)
```

```
[[1.  0.  0.  0.  0.  0.  0.  0.  0. ]
 [0.333 1.  0.  0.  0.  0.  0.  0.  0. ]
 [0.111 0.333 1.  0.  0.  0.  0.  0.  0. ]
 [0.037 0.111 0.333 1.  0.  0.  0.  0.  0. ]
 [0.012 0.037 0.111 0.333 1.  0.  0.  0.  0. ]
 [0.004 0.012 0.037 0.111 0.333 1.  0.  0.  0. ]
 [0.001 0.004 0.012 0.037 0.111 0.333 1.  0.  0. ]
 [0.  0.001 0.004 0.012 0.037 0.111 0.333 1.  0. ]
 [0.  0.  0.001 0.004 0.012 0.037 0.111 0.333 1. ]]
```

Try to speed up by computing the inverse

A unit lower triangular matrix has a unit lower triangular inverse

```
1 np.set_printoptions(precision=3, suppress=True)
2 Ai = np.linalg.inv(A.toarray())
```

Now solve directly using A^{-1}

```
1 ui = Ai @ b
```

Compare with previous - still the same

```
1 um = sparse.linalg.spsolve(A, b)
2 print(um-ui)
```

```
[0. 0. 0. 0. 0. 0. 0. 0. 0.]
```

but it seems to be very fast

```
1 %timeit -n 100 ui = Ai @ b
```

```
601 ns ± 76.3 ns per loop (mean ± std. dev. of 7 runs, 100 loops each)
```

Until we try Numba on the explicit solver

```
1 import numba as nb
2
3 @nb.jit
4 def nbreccsolve(u, N, I, g):
5     u[0] = I
6     for n in range(N):
7         u[n+1] = g * u[n]
8     nbreccsolve(u, N, I, g)
9 %timeit -n 100 nbreccsolve(u, N, I, g)
```

288 ns $\pm$ 34.9 ns per loop (mean $\pm$ std. dev. of 7 runs, 100 loops each)

Bottom line - explicit marching methods are very fast! Unfortunately, they may be unstable and cannot be used for BVPs.

The BV vibration problem

$$u'' + \omega^2 u = 0, t \in (0, T) \quad u(0) = I, u(T) = I,$$

cannot be solved with recurrence, since it is not an initial value problem.

However, we can solve this problem using a central finite difference for all internal points $n = 1, 2, \dots, N - 1$

$$\frac{u^{n+1} - 2u^n + u^{n-1}}{\Delta t^2} + \omega^2 u^n = 0.$$

This leads to an **implicit** linear algebra problem, because the equation for u^n (above) depends on u^{n+1} !

Vibration on matrix form

The matrix problem is now, using $g = 2 - \omega^2 \Delta t^2$,

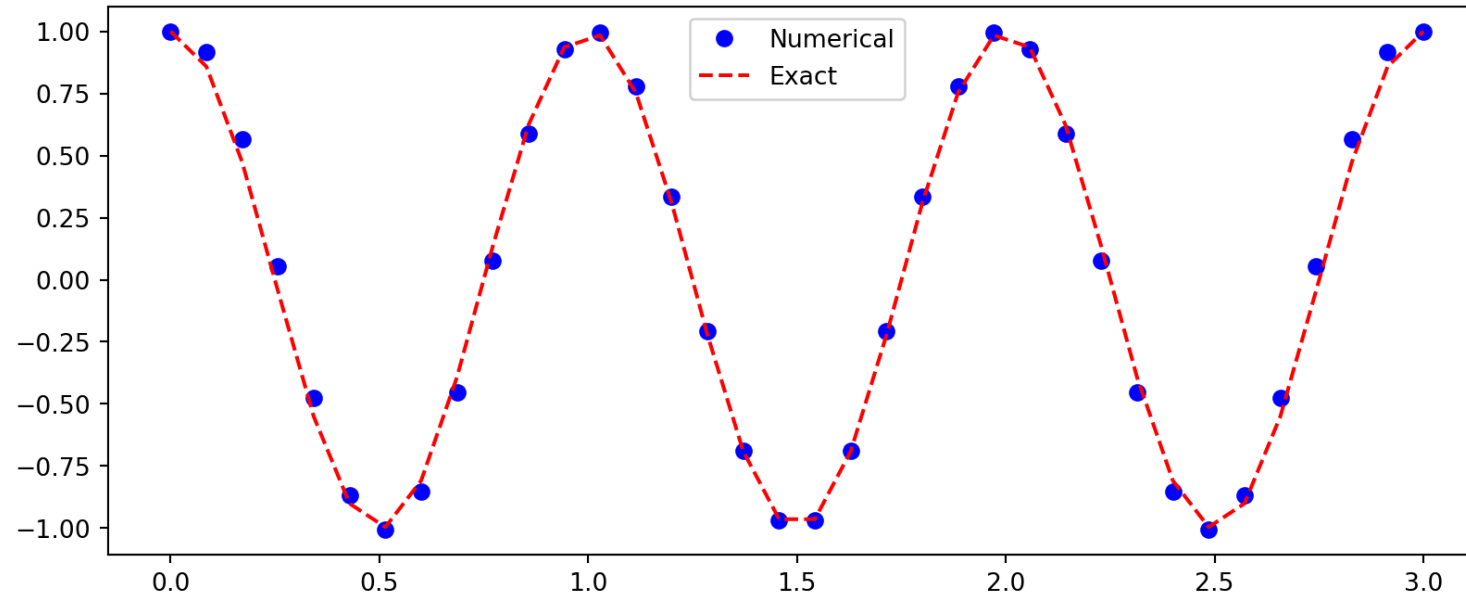
$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & -g & 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & -g & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & -g & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & -g & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & -g & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} u^0 \\ u^1 \\ u^2 \\ u^3 \\ u^4 \\ u^5 \\ u^6 \end{bmatrix} = \begin{bmatrix} I \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ I \end{bmatrix}$$

Note

- The matrix contains items both over and under the main diagonal, which characterizes an **implicit** method. **Explicit** marching method leads to lower triangular matrices.
- The first and last rows are modified in order to apply boundary conditions **implicitly**. More about that later.

Assemble and solve the implicit BVP

```
1 T, N, I, w = 3., 35, 1., 2*np.pi
2 dt = T/N
3 g = 2 - w**2*dt**2
4 A = sparse.diags([1, -g, 1], np.array([-1, 0, 1]), (N+1, N+1), 'lil')
5 b = np.zeros(N+1)
6 A[0, :3] = 1, 0, 0 # Fix first row
7 A[-1, -3:] = 0, 0, 1 # Fix last row
8 b[0], b[-1] = I, I
9 u2 = sparse.linalg.spsolve(A.tocsr(), b)
10 t = np.linspace(0, T, N+1)
11 plt.figure(figsize=(10, 4))
12 plt.plot(t, u2, 'bo', t, I*np.cos(w*t), 'r--')
13 plt.legend(['Numerical', 'Exact']);
```



Implicit finite difference methods

Taylor expansions can be used to design finite difference methods of any order

For example, we can create two backward and two forward Taylor expansions starting from $u^n = u(t_n)$

$$(-2) \quad u^{n-2} = u^n - 2hu' + \frac{2h^2}{1}u'' - \frac{4h^3}{3}u''' + \frac{2h^4}{3}u'''' - \dots$$

$$(-1) \quad u^{n-1} = u^n - hu' + \frac{h^2}{2}u'' - \frac{h^3}{6}u''' + \frac{h^4}{24}u'''' - \dots$$

$$(1) \quad u^{n+1} = u^n + hu' + \frac{h^2}{2}u'' + \frac{h^3}{6}u''' + \frac{h^4}{24}u'''' + \dots$$

$$(2) \quad u^{n+2} = u^n + 2hu' + \frac{2h^2}{1}u'' + \frac{4h^3}{3}u''' + \frac{2h^4}{3}u'''' + \dots$$

Remember: $u^{n+a} = u(t_{n+a})$ and $t_{n+a} = (n+a)h$ and we use $h = \Delta t$ for simplicity.

Add equations (-1) and (1) and isolate $u''(t_n)$

$$u''(t_n) = \frac{u^{n+1} - 2u^n + u^{n-1}}{h^2} + \frac{h^2}{12}u'''' +$$

The FD stencil can be applied to the entire mesh

That is, we can compute $\mathbf{u}^{(2)} = (u''(t_n))_{n=0}^N$ from the mesh function $\mathbf{u} = (u(t_n))_{n=0}^N \in \mathbb{R}^{N+1}$ as

$$\mathbf{u}^{(2)} = D^{(2)} \mathbf{u},$$

where $D^{(2)} \in \mathbb{R}^{N+1 \times N+1}$ is the second **derivative matrix**.

$$\underbrace{\begin{bmatrix} u_0^{(2)} \\ u_1^{(2)} \\ u_2^{(2)} \\ \vdots \\ u_{N-2}^{(2)} \\ u_{N-1}^{(2)} \\ u_N^{(2)} \end{bmatrix}}_{\mathbf{u}^{(2)}} = \frac{1}{h^2} \underbrace{\begin{bmatrix} ? & ? & ? & ? & ? & ? & ? & ? \\ 1 & -2 & 1 & 0 & 0 & 0 & 0 & \dots \\ 0 & 1 & -2 & 1 & 0 & 0 & 0 & \dots \\ \vdots & & & \ddots & & & & \vdots \\ \dots & 0 & 0 & 0 & 1 & -2 & 1 & 0 \\ \dots & 0 & 0 & 0 & 0 & 1 & -2 & 1 \\ ? & ? & ? & ? & ? & ? & ? & ? \end{bmatrix}}_{D^{(2)}} \underbrace{\begin{bmatrix} u^0 \\ u^1 \\ u^2 \\ \vdots \\ u^{N-1} \\ u^{N-1} \\ u^N \end{bmatrix}}_{\mathbf{u}}$$

How about the boundary nodes?

$$\underbrace{\begin{bmatrix} u_0^{(2)} \\ u_1^{(2)} \\ u_2^{(2)} \\ \vdots \\ u_{N-2}^{(2)} \\ u_{N-1}^{(2)} \\ u_N^{(2)} \end{bmatrix}}_{\mathbf{u}^{(2)}} = \frac{1}{h^2} \underbrace{\begin{bmatrix} ? & ? & ? & ? & ? & ? & ? & ? \\ 1 & -2 & 1 & 0 & 0 & 0 & 0 & \dots \\ 0 & 1 & -2 & 1 & 0 & 0 & 0 & \dots \\ \vdots & & & \ddots & & & & \vdots \\ \dots & 0 & 0 & 0 & 1 & -2 & 1 & 0 \\ \dots & 0 & 0 & 0 & 0 & 1 & -2 & 1 \\ ? & ? & ? & ? & ? & ? & ? & ? \end{bmatrix}}_{D^{(2)}} \underbrace{\begin{bmatrix} u^0 \\ u^1 \\ u^2 \\ \vdots \\ u^{N-1} \\ u^{N-1} \\ u^N \end{bmatrix}}_{\mathbf{u}}$$

What to do with the first and last rows? The central stencils do not work here.

We still want to compute $u_0^{(2)} = u''(t_0)$ and $u_N^{(2)} = u''(t_N)$. How?

Forward and backward stencils will do

We can do a forward difference at $n = 0$

Remember:

$$(1) \quad u^{n+1} = u^n + hu' + \frac{h^2}{2}u'' + \frac{h^3}{6}u''' + \frac{h^4}{24}u'''' + \dots$$

$$(2) \quad u^{n+2} = u^n + 2hu' + \frac{2h^2}{1}u'' + \frac{4h^3}{3}u''' + \frac{2h^4}{3}u'''' + \dots$$

Subtract 2 times Eq. (1) from Eq. (2) and rearrange

$$(2) - 2(1) : u^{n+2} - 2u^{n+1} = -u^n + \frac{h^2}{1}u'' + h^3u''' + \frac{7h^4}{12}u'''' +$$

Rearrange to isolate a **first order** accurate stencil for $u''(0)$

$$u''(0) = \frac{u^2 - 2u^1 + u^0}{h^2} - hu'''(0) - \frac{7h^2}{12}u''''(0) +$$

We can do a backward difference at $n = N$

Use two backward Taylor expansions:

$$(-2) \quad u^{n-2} = u^n - 2hu' + \frac{2h^2}{1}u'' - \frac{4h^3}{3}u''' + \frac{2h^4}{3}u'''' - \dots$$

$$(-1) \quad u^{n-1} = u^n - hu' + \frac{h^2}{2}u'' - \frac{h^3}{6}u''' + \frac{h^4}{24}u'''' - \dots$$

Subtract 2 times Eq. (-1) from Eq. (-2) and rearrange

$$(-2) - 2(-1) : u^{n-2} - 2u^{n-1} = -u^n + \frac{h^2}{1}u'' - h^3u''' + \frac{7h^4}{12}u'''' +$$

Rearrange to isolate a **first order** accurate backward stencil for $u''(T)$

$$u''(T) = \frac{u^{N-2} - 2u^{N-1} + u^N}{h^2} - hu'''(T) - \frac{7h^2}{12}u''''(T) +$$

We get a second derivative matrix

$$D^{(2)} = \frac{1}{h^2} \begin{bmatrix} 1 & -2 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & -2 & 1 & 0 & 0 & 0 & 0 & \dots \\ 0 & 1 & -2 & 1 & 0 & 0 & 0 & \dots \\ \vdots & & & \ddots & & & & \vdots \\ \dots & 0 & 0 & 0 & 1 & -2 & 1 & 0 \\ \dots & 0 & 0 & 0 & 0 & 1 & -2 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & -2 & 1 \end{bmatrix}$$

but with merely first order accuracy at the boundaries. Can we do better?

Yes! Of course, just use more points near the boundaries!

Use three forward points $u^{n+1}, u^{n+2}, u^{n+3}$ to get a second order forward stencil

$$(1) \quad u^{n+1} = u^n + hu' + \frac{h^2}{2}u'' + \frac{h^3}{6}u''' + \frac{h^4}{24}u'''' + \dots$$

$$(2) \quad u^{n+2} = u^n + 2hu' + \frac{2h^2}{1}u'' + \frac{4h^3}{3}u''' + \frac{2h^4}{3}u'''' + \dots$$

$$(3) \quad u^{n+3} = u^n + 3hu' + \frac{9h^2}{2}u'' + \frac{9h^3}{2}u''' + \frac{27h^4}{8}u'''' + \dots$$

Now to eliminate both u' and u''' terms add the three equations as $-(3) + 4 \cdot (2) - 5 \cdot (1)$ (don't worry about how I know this yet)

$$-(3) + 4 \cdot (2) - 5 \cdot (1) : -u^{n+3} + 4u^{n+2} - 5u^{n+1} = -2u^n + h^2u'' - \frac{11h^4}{12}u'''' +$$

Isolate $u''(0)$

$$u''(0) = \frac{-u^3 + 4u^2 - 5u^1 + 2u^0}{h^2} + \frac{11h^2}{12}u'''' +$$

Fully second order $D^{(2)}$

$$D^{(2)} = \frac{1}{h^2} \begin{bmatrix} 2 & -5 & 4 & -1 & 0 & 0 & 0 & 0 \\ 1 & -2 & 1 & 0 & 0 & 0 & 0 & \dots \\ 0 & 1 & -2 & 1 & 0 & 0 & 0 & \dots \\ \vdots & & & \ddots & & & & \vdots \\ \dots & 0 & 0 & 0 & 1 & -2 & 1 & 0 \\ \dots & 0 & 0 & 0 & 0 & 1 & -2 & 1 \\ 0 & 0 & 0 & 0 & -1 & 4 & -5 & 2 \end{bmatrix}$$

Assemble in Python

```
1 N = 8
2 dt = 0.5
3 T = N*dt
4 D2 = sparse.diags([1, -2, 1], np.array([-1, 0, 1]), (N+1, N+1), 'lil')
5 D2[0, :4] = 2, -5, 4, -1
6 D2[-1, -4:] = -1, 4, -5, 2
7 D2 *= (1/dt**2) # don't forget h
8 D2.toarray()*dt**2
```

```
array([[ 2., -5.,  4., -1.,  0.,  0.,  0.,  0.,  0.],
       [ 1., -2.,  1.,  0.,  0.,  0.,  0.,  0.,  0.],
       [ 0.,  1., -2.,  1.,  0.,  0.,  0.,  0.,  0.],
       [ 0.,  0.,  1., -2.,  1.,  0.,  0.,  0.,  0.],
       [ 0.,  0.,  0.,  1., -2.,  1.,  0.,  0.,  0.],
       [ 0.,  0.,  0.,  0.,  1., -2.,  1.,  0.,  0.],
       [ 0.,  0.,  0.,  0.,  0.,  1., -2.,  1.,  0.],
       [ 0.,  0.,  0.,  0.,  0.,  0.,  1., -2.,  1.],
       [ 0.,  0.,  0.,  0.,  0., -1.,  4., -5.,  2.]])
```

Apply matrix to a mesh function $f(t_n) = t_n^2$

```
1 t = np.linspace(0, N*dt, N+1)
2 f = t**2
3 D2 @ f
```

```
array([2.,  2.,  2.,  2.,  2.,  2.,  2.,  2.,  2.])
```

Exact for all n !

Try with merely first order boundary

```
1 D21 = sparse.diags([1, -2, 1], np.array([-1, 0, 1]), (N+1, N+1), 'lil')
2 D21[0, :4] = 1, -2, 1, 0
3 D21[-1, -4:] = 0, 1, -2, 1
4 D21 *= (1/dt**2)
5 D21 @ f
```

```
array([2., 2., 2., 2., 2., 2., 2., 2., 2.])
```

Still exact! Why?

Consider the forward first order stencil

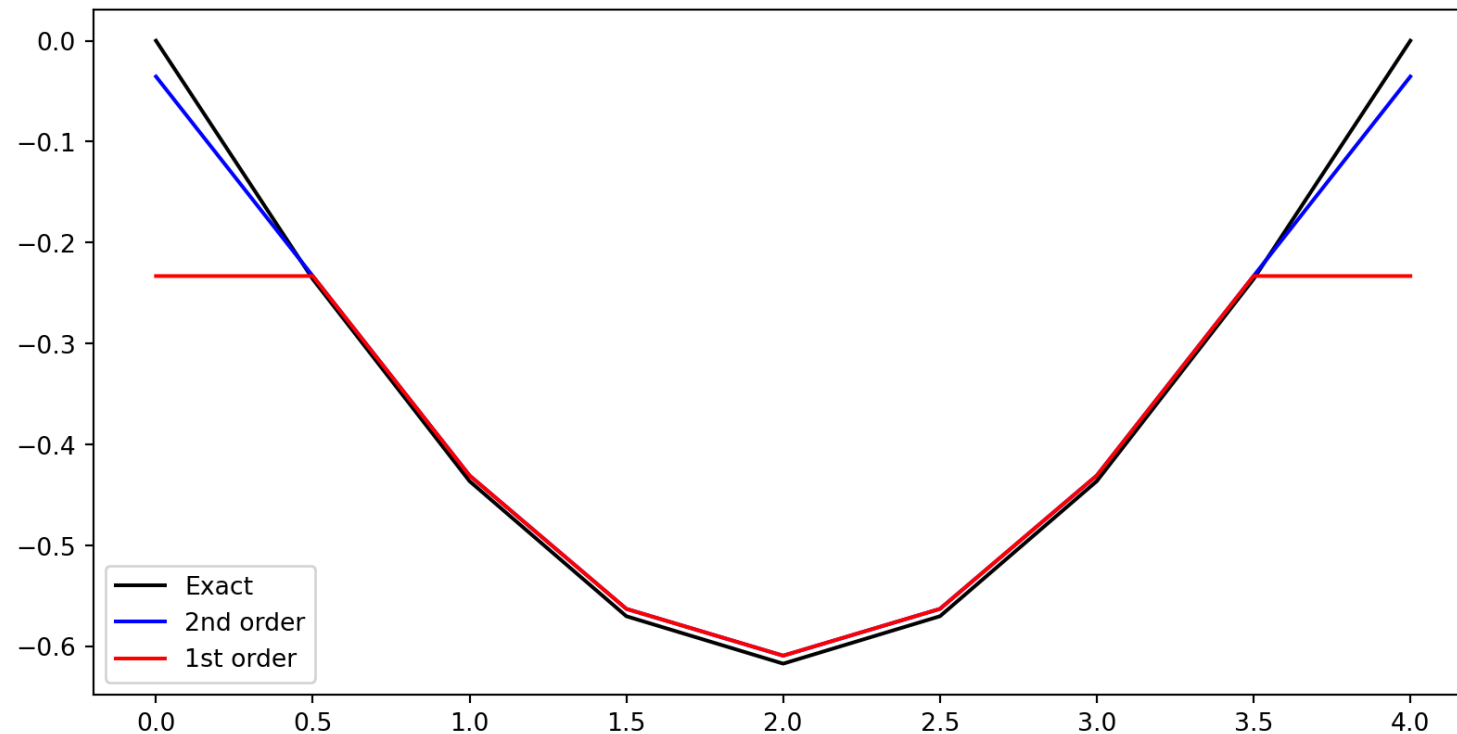
$$u''(0) = \frac{u^2 - 2u^1 + u^0}{h^2} - hu'''(0) - \frac{7h^2}{12}u''''(0) +$$

The leading error is $hu'''(0)$ and $u'''(0)$ is zero for $u(t) = t^2$. The second order stencil captures the second order polynomial t^2 exactly!

More challenging example

Let $f(t) = \sin(\pi t/T)$ such that $f''(t) = -(\pi/T)^2 f(t)$. Here none of the error terms will disappear and we see the effect of the poor first order boundary.

```
1 f = np.sin(np.pi*t / T)
2 d2fe = -(np.pi/T)**2*f
3 d2f = D2 @ f
4 d2f1 = D21 @ f
5 plt.plot(t, d2fe, 'k', t, d2f, 'b', t, d2f1, 'r')
6 plt.legend(['Exact', '2nd order', '1st order']);
```



First derivative matrix

Lets create a similar matrix for a second order accurate single derivative.

$$\begin{aligned}(-1) \quad u^{n-1} &= u^n - hu' + \frac{h^2}{2}u'' - \frac{h^3}{6}u''' + \frac{h^4}{24}u'''' + \dots \\(1) \quad u^{n+1} &= u^n + hu' + \frac{h^2}{2}u'' + \frac{h^3}{6}u''' + \frac{h^4}{24}u'''' + \dots\end{aligned}$$

Here Eq. (1) minus Eq. (-1) leads to

$$u'(t_n) = \frac{u^{n+1} - u^{n-1}}{2h} + \frac{h^2}{6}u''' +$$

which is second order accurate.

The central scheme cannot be used for $n = 0$ or $n = N$.

We use a forward scheme for $n = 0$

$$(1) \quad u^{n+1} = u^n + hu' + \frac{h^2}{2}u'' + \frac{h^3}{6}u''' + \frac{h^4}{24}u^{(4)} + \dots$$

$$(2) \quad u^{n+2} = u^n + 2hu' + \frac{2h^2}{1}u'' + \frac{4h^3}{3}u''' + \frac{2h^4}{3}u^{(4)} + \dots$$

From Eq. (1):

$$u'(t_n) = \frac{u^{n+1} - u^n}{h} - \frac{h}{2}u'' -$$

Adding one more equation (2) we get second order: $4 \cdot (1) - (2)$ (Note that the terms with u'' then cancel)

$$u'(t_n) = \frac{-u^{n+2} + 4u^{n+1} - 3u^n}{2h} + \frac{h^2}{3}u''' +$$

With forward and backward on the edges we get a second order first derivative matrix

$$D^{(1)} = \frac{1}{2h} \begin{bmatrix} -3 & 4 & -1 & 0 & 0 & 0 & 0 & 0 \\ -1 & 0 & 1 & 0 & 0 & 0 & 0 & \dots \\ 0 & -1 & 0 & 1 & 0 & 0 & 0 & \dots \\ \vdots & & & \ddots & & & & \vdots \\ \dots & 0 & 0 & 0 & -1 & 0 & 1 & 0 \\ \dots & 0 & 0 & 0 & 0 & -1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & -4 & 3 \end{bmatrix}$$

```
1 D1 = sparse.diags([-1, 1], np.array([-1, 1]), (N+1, N+1), 'lil')
2 D1[0, :3] = -3, 4, -1
3 D1[-1, -3:] = 1, -4, 3
4 D1 *= (1/(2*dt))
5 f = t
6 D1 @ f
```

```
array([1., 1., 1., 1., 1., 1., 1., 1., 1.])
```

Is $D^{(2)}$ the same as $D^{(1)} D^{(1)}$?

Is it the same to compute $D^{(2)}\mathbf{u}$ as $D^{(1)}(D^{(1)}\mathbf{u})$?

Analytically, we know that $u'' = \frac{d^2 u}{dt^2} = \frac{d}{dt} \left(\frac{du}{dt} \right)$, but this is not necessarily so numerically.

```
1 D22 = D1 @ D1 # @ represents matrix-matrix product
2 D22.toarray()*4*dt**2
```

```
array([[ 5., -11.,  7., -1.,  0.,  0.,  0.,  0.,  0.],
       [ 3., -5.,  1.,  1.,  0.,  0.,  0.,  0.,  0.],
       [ 1.,  0., -2.,  0.,  1.,  0.,  0.,  0.,  0.],
       [ 0.,  1.,  0., -2.,  0.,  1.,  0.,  0.,  0.],
       [ 0.,  0.,  1.,  0., -2.,  0.,  1.,  0.,  0.],
       [ 0.,  0.,  0.,  1.,  0., -2.,  0.,  1.,  0.],
       [ 0.,  0.,  0.,  0.,  1.,  0., -2.,  0.,  1.],
       [ 0.,  0.,  0.,  0.,  0.,  1.,  1., -5.,  3.],
       [ 0.,  0.,  0.,  0.,  0., -1.,  7., -11.,  5.]])
```

This stencil is wider than $D^{(2)}$, using neighboring points farther away. The central stencil is

$$u''(t_n) = \frac{u^{n+2} - 2u^n + u^{n-2}}{4h^2}$$

How accurate is $D^{(1)} D^{(1)}$?

Internal points

$$(-2) \quad u^{n-2} = u^n - 2hu' + \frac{2h^2}{1}u'' - \frac{4h^3}{3}u''' + \frac{2h^4}{3}u'''' - \dots$$

$$(2) \quad u^{n+2} = u^n + 2hu' + \frac{2h^2}{1}u'' + \frac{4h^3}{3}u''' + \frac{2h^4}{3}u'''' + \dots$$

Take Eq. (2) plus Eq. (-2)

$$u^{n+2} + u^{n-2} = 2u^n + 4h^2u'' + \frac{4h^4}{3}u'''' +$$

and

$$u''(t_n) = \frac{u^{n+2} - 2u^n + u^{n-2}}{4h^2} - \frac{h^2}{3}u'''' + \dots$$

The stencil in the center (for $1 < n < N - 1$) of $D^{(1)}D^{(1)}$ is second order!

How about boundaries?

We have obtained $u''(t_1) = \frac{u^{n+2} + u^{n+1} - 5u^n + 3u^{n-1}}{4h^2}$. Compute the error in this stencil using one backward and two forward points

$$(-1) \quad u^{n-1} = u^n - hu' + \frac{h^2}{2}u'' - \frac{h^3}{6}u''' + \frac{h^4}{24}u'''' - \dots$$

$$(1) \quad u^{n+1} = u^n + hu' + \frac{h^2}{2}u'' + \frac{h^3}{6}u''' + \frac{h^4}{24}u'''' + \dots$$

$$(2) \quad u^{n+2} = u^n + 2hu' + \frac{2h^2}{1}u'' + \frac{4h^3}{3}u''' + \frac{2h^4}{3}u'''' + \dots$$

Take Eq. (2) + (1) + 3 · (-1)

$$u''(t_1) = \frac{u^{n+2} + u^{n+1} - 5u^n + 3u^{n-1}}{4h^2} + \frac{h}{4}u''' + \dots$$

Only first order.

How about the border point for $n = 0$?

We observe that $u''(t_0) = \frac{-u^{n+3} + 7u^{n+2} - 11u^{n+1} + 5u^n}{4h^2}$

$$(1) \quad u^{n+1} = u^n + hu' + \frac{h^2}{2}u'' + \frac{h^3}{6}u''' + \frac{h^4}{24}u'''' + \dots$$

$$(2) \quad u^{n+2} = u^n + 2hu' + \frac{2h^2}{1}u'' + \frac{4h^3}{3}u''' + \frac{2h^4}{3}u'''' + \dots$$

$$(3) \quad u^{n+3} = u^n + 3hu' + \frac{9h^2}{2}u'' + \frac{9h^3}{2}u''' + \frac{27h^4}{8}u'''' + \dots$$

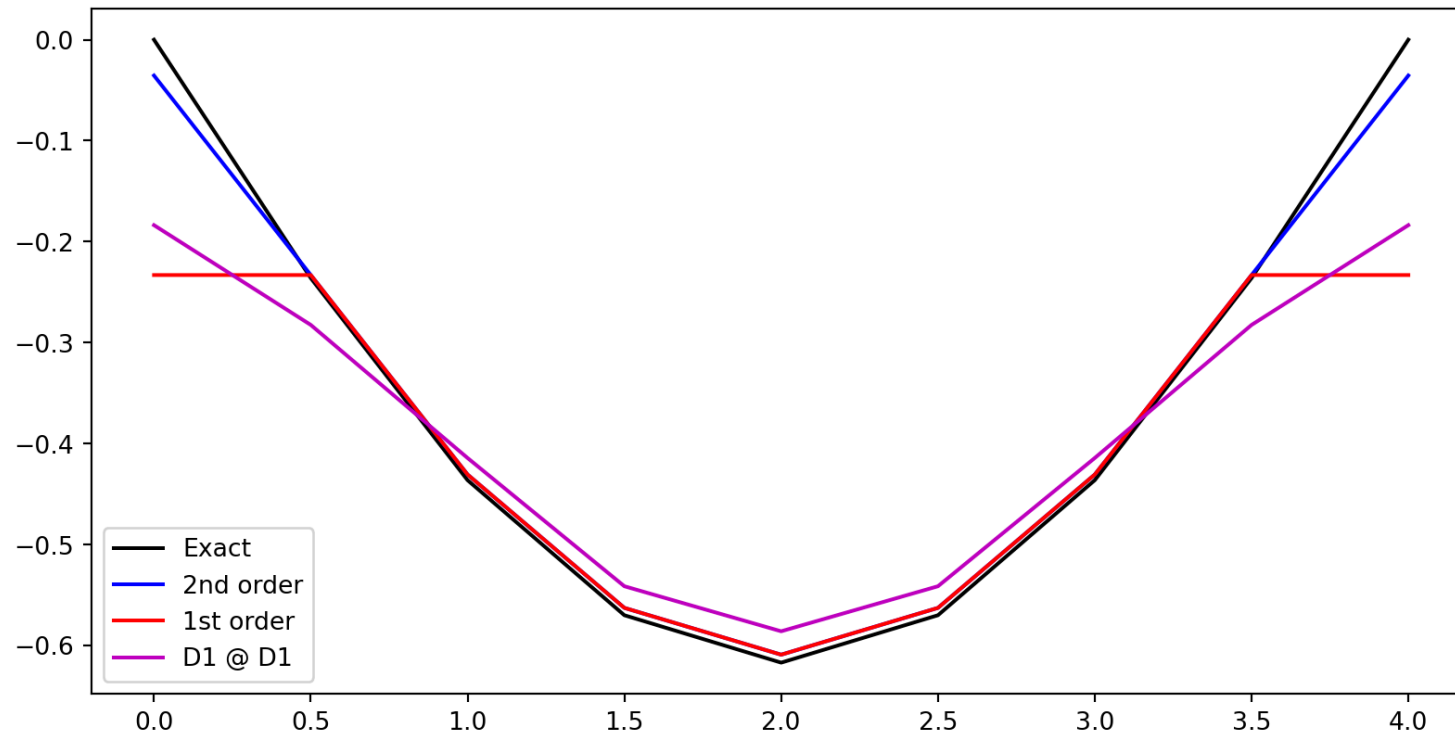
Take Eq. $-(3) + 7 \cdot (2) - 11 \cdot (1)$ to obtain

$$u''(t_0) = \frac{-u^{n+3} + 7u^{n+2} - 11u^{n+1} + 5u^n}{4h^2} - \frac{3h}{4}u''' \dots$$

First order. So we conclude that $D^{(1)}D^{(1)}$ is still an approximation of the second derivative, but it is only first order accurate near the boundaries.

Test the accuracy of $D^{(2)}$, $D^{(2)}$ with only first order boundary and $D^{(1)} D^{(1)}$

```
1 f = np.sin(np.pi*t / T)
2 d2fe = -(np.pi/T)**2*f
3 d2f = D2 @ f
4 d2f1 = D21 @ f
5 d2f2 = D1 @ D1 @ f
6 plt.plot(t, d2fe, 'k', t, d2f, 'b', t, d2f1, 'r', t, d2f2, 'm')
7 plt.legend(['Exact', '2nd order', '1st order', 'D1 @ D1']);
```



The main advantage of using difference matrices is that they make it easy to solve equations

1. Discretize $\rightarrow$ Simply replace k 'th derivative in the ODE with the k 'th derivative matrix
2. Apply boundary conditions to the assembled matrix

Consider the exponential decay model

$$u' + au = 0, \quad t \in (0, T], \quad u(0) = I.$$

Replace derivatives with derivative matrices ($u' = D^{(1)}\mathbf{u}$, $u = \mathbb{I}\mathbf{u}$):

$$(D^{(1)} + a\mathbb{I})\mathbf{u} = \mathbf{0},$$

where $\mathbb{I} \in \mathbb{R}^{N+1 \times N+1}$ is the identity matrix and $\mathbf{0} \in \mathbb{R}^{N+1}$ is a null-vector. We get

$$A\mathbf{u} = \mathbf{0}, \quad A = D^{(1)} + a\mathbb{I}$$

which is trivially solved to $\mathbf{u} = \mathbf{0}$ before adding boundary conditions. Not after.

Apply boundary condition by modifying both the coefficient matrix A and the rhs vector b

With just the derivative matrix we have the problem

$$Au = b, \quad \text{for } b \in \mathbb{R}^{N+1}$$

$$\underbrace{\frac{1}{2h} \begin{bmatrix} -3 + 2ah & 4 & -1 & 0 & 0 & 0 & 0 & 0 \\ -1 & 2ah & 1 & 0 & 0 & 0 & 0 & \dots \\ 0 & -1 & 2ah & 1 & 0 & 0 & 0 & \dots \\ \vdots & & & \ddots & & & \vdots & \\ \dots & 0 & 0 & 0 & -1 & 2ah & 1 & 0 \\ \dots & 0 & 0 & 0 & 0 & -1 & -2ah & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & -4 & 3 + 2ah \end{bmatrix}}_{D^{(1)} + a\mathbb{I}} \underbrace{\begin{bmatrix} u^0 \\ u^1 \\ u^2 \\ \vdots \\ u^{N-2} \\ u^{N-1} \\ u^N \end{bmatrix}}_u = \underbrace{\begin{bmatrix} b^0 \\ b^1 \\ b^2 \\ \vdots \\ b^{N-2} \\ b^{N-1} \\ b^N \end{bmatrix}}_b =$$

In order to enforce that $u(0) = u^0 = I$, we can modify the first row of the coefficient matrix A and the right hand side vector b

Modify A and b to enforce boundary conditions

$$\underbrace{\frac{1}{2h} \begin{bmatrix} 2h & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ -1 & 2ah & 1 & 0 & 0 & 0 & 0 & \dots \\ 0 & -1 & 2ah & 1 & 0 & 0 & 0 & \dots \\ \vdots & & & \ddots & & & \vdots & \\ \dots & 0 & 0 & 0 & -1 & 2ah & 1 & 0 \\ \dots & 0 & 0 & 0 & 0 & -1 & -2ah & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & -4 & 3 + 2ah \end{bmatrix}}_A \underbrace{\begin{bmatrix} u^0 \\ u^1 \\ u^2 \\ \vdots \\ u^{N-2} \\ u^{N-1} \\ u^N \end{bmatrix}}_u = \underbrace{\begin{bmatrix} I \\ 0 \\ 0 \\ \vdots \\ 0 \\ 0 \\ 0 \end{bmatrix}}_b$$

Now the equation in the first row states that $u^0 = I$, whereas the remaining rows are unchanged and solve the central and **implicit** problem for row n (row N is slightly different)

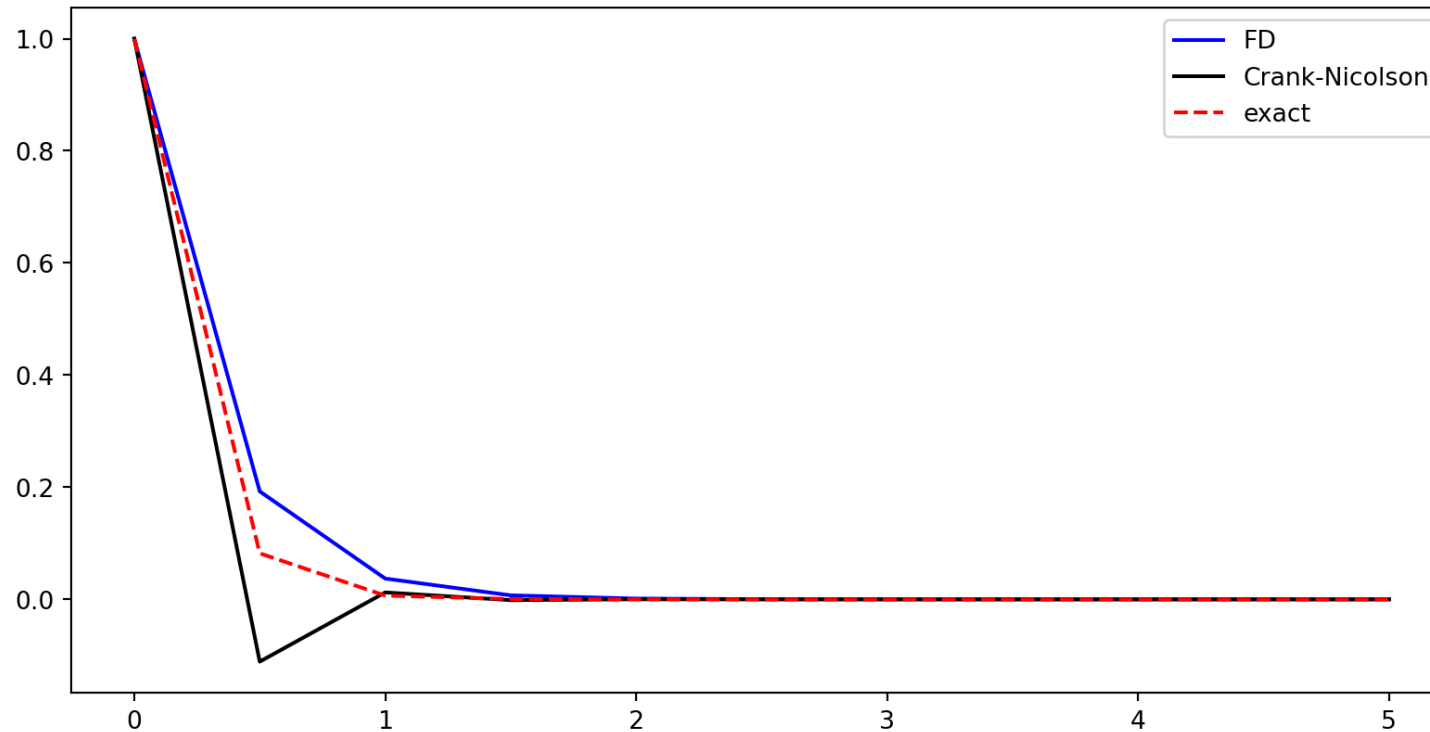
$$\frac{u^{n+1} - u^{n-1}}{2h} + au^n = 0$$

Implement decay problem

```
1 N = 10
2 dt = 0.5
3 a = 5
4 T = N*dt
5 t = np.linspace(0, N*dt, N+1)
6 D1 = sparse.diags([-1, 1], np.array([-1, 1]), (N+1, N+1), 'lil')
7 D1[0, :3] = -3, 4, -1
8 D1[-1, -3:] = 1, -4, 3
9 D1 *= (1/(2*dt))
10 Id = sparse.eye(N+1)
11 A = D1 + a*Id
12 b = np.zeros(N+1)
13 b[0] = 1
14 A[0, :3] = 1, 0, 0 # boundary condition
15 A.toarray()
```

```
array([[ 1.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.],
       [-1.,  5.,  1.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.],
       [ 0., -1.,  5.,  1.,  0.,  0.,  0.,  0.,  0.,  0.,  0.],
       [ 0.,  0., -1.,  5.,  1.,  0.,  0.,  0.,  0.,  0.,  0.],
       [ 0.,  0.,  0., -1.,  5.,  1.,  0.,  0.,  0.,  0.,  0.],
       [ 0.,  0.,  0.,  0., -1.,  5.,  1.,  0.,  0.,  0.,  0.],
       [ 0.,  0.,  0.,  0.,  0., -1.,  5.,  1.,  0.,  0.,  0.],
       [ 0.,  0.,  0.,  0.,  0.,  0., -1.,  5.,  1.,  0.,  0.],
       [ 0.,  0.,  0.,  0.,  0.,  0.,  0., -1.,  5.,  1.,  0.],
       [ 0.,  0.,  0.,  0.,  0.,  0.,  0.,  0., -1.,  5.,  1.],
       [ 0.,  0.,  0.,  0.,  0.,  0.,  0.,  0.,  1., -4.,  8.]])
```

Compare with exact solution



The accuracy is similar to the Crank-Nicolson method discussed in [lectures 1-2](#). However, the FD method is not a recursive **marching method**, since the equation for u^n depends on the solution at u^{n+1} ! This **implicit** FD method is unconditionally stable. Normally, only marching methods are analysed for stability.

The vibration eq. is often considered as a BVP

$$u''(t) + \omega^2 u(t) = 0, \quad u(0) = u(T) = I, \quad t = (0, T)$$

Discretize by replacing the derivative with a second differentiation matrix

$$(D^{(2)} + \omega^2 \mathbb{I})\mathbf{u} = \mathbf{b}$$

```
1 w = 2*np.pi
2 T, N, I = 3., 35, 1.
3 dt = T/N
4 D2 = sparse.diags([1, -2, 1], np.array([-1, 0, 1]), (N+1, N+1), 'lil')
5 D2 *= (1/(dt**2)) # never mind the edges n=0 or n=N because these rows will be overwritten
6 Id = sparse.eye(N+1)
7 A = D2 + w**2*Id
8 b = np.zeros(N+1)
9 A.toarray()*dt**2
```

```
array([[ -1.71,  1.   ,  0.   , ...,  0.   ,  0.   ,  0.   ],
       [  1.   , -1.71,  1.   , ...,  0.   ,  0.   ,  0.   ],
       [  0.   ,  1.   , -1.71, ...,  0.   ,  0.   ,  0.   ],
       ...,
       [  0.   ,  0.   ,  0.   , ..., -1.71,  1.   ,  0.   ],
       [  0.   ,  0.   ,  0.   , ...,  1.   , -1.71,  1.   ],
       [  0.   ,  0.   ,  0.   , ...,  0.   ,  1.   , -1.71]], shape=(36, 36))
```


Apply boundary conditions to the first and last rows

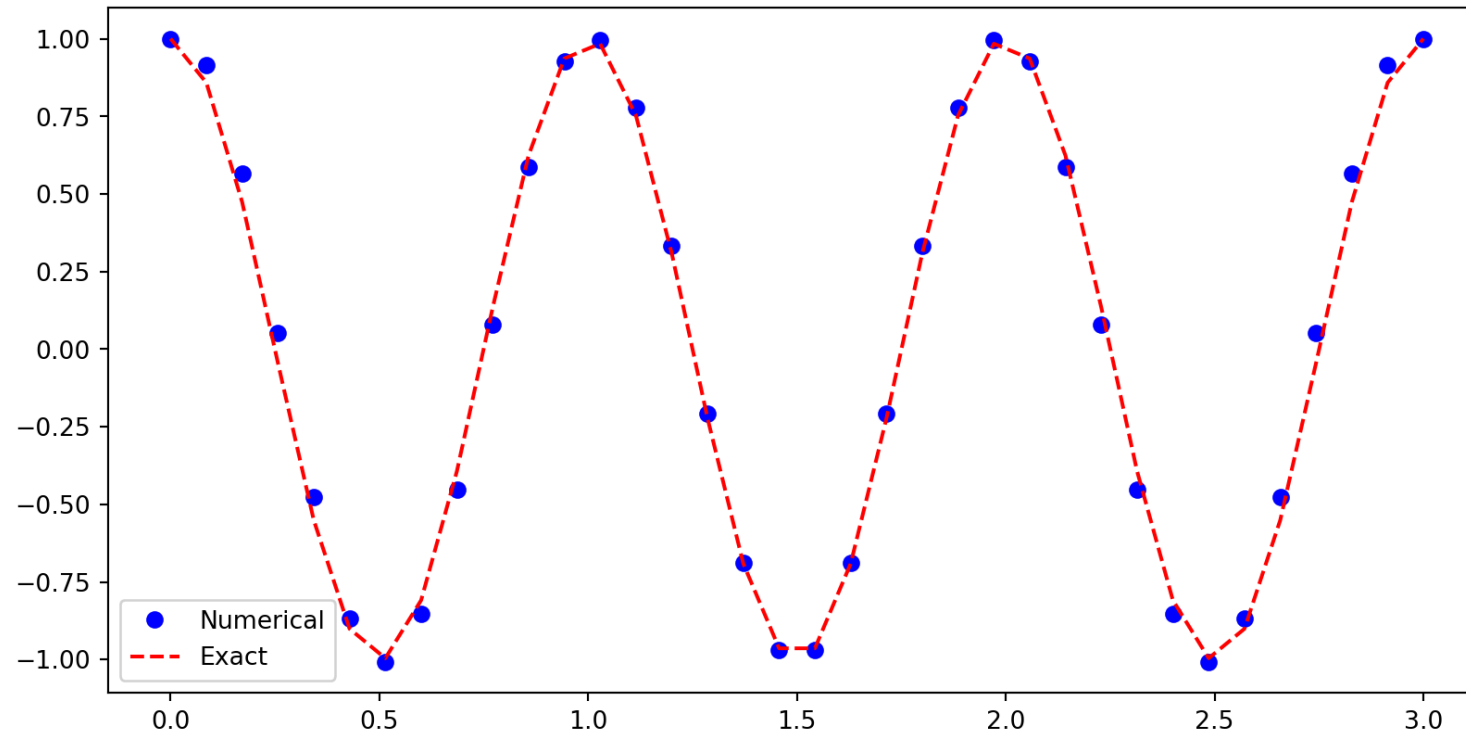
$$u^0 = I$$
$$u^N = I$$

```
1 A[0, :3] = 1, 0, 0
2 A[-1, -3:] = 0, 0, 1
3 b[0] = I
4 b[-1] = I
5 A.toarray()
```

```
array([[ 1.    ,  0.    ,  0.    , ...,  0.    ,  0.    ,  0.    ],
       [136.111, -232.744, 136.111, ...,  0.    ,  0.    ,  0.    ],
       [ 0.    , 136.111, -232.744, ...,  0.    ,  0.    ,  0.    ],
       ...,
       [ 0.    ,  0.    ,  0.    , ..., -232.744, 136.111,  0.    ],
       [ 0.    ,  0.    ,  0.    , ..., 136.111, -232.744, 136.111],
       [ 0.    ,  0.    ,  0.    , ...,  0.    ,  0.    ,  1.    ]],
      shape=(36, 36))
```

Solve the vibration equation

```
1 u2 = sparse.linalg.spsolve(A, b)
2 t = np.linspace(0, T, N+1)
3 plt.plot(t, u2, 'bo', t, I*np.cos(w*t), 'r--')
4 plt.legend(['Numerical', 'Exact'])
```



Generic finite difference stencils

We have seen that it is quite simple to develop finite difference stencils for any derivative, using either forward or backward points. Can this be generalized?

Yes! Of course it can. The generic Taylor expansion around x_0 reads

$$u(x) = \sum_{i=0}^M \frac{(x - x_0)^i}{i!} u^{(i)}(x_0) + \mathcal{O}((x - x_0)^{M+1}),$$

where $u^{(i)}(x_0) = \frac{d^i u}{dx^i} \big|_{x=x_0}$ and there are $M + 1$ terms in the expansion.

Use only $x = x_0 + ph$, where p is an integer and h is a constant (Δt or Δx)

$$u^{n+p} = \sum_{i=0}^M \frac{(ph)^i}{i!} u^{(i)}(x_0) + \mathcal{O}(h^{M+1}),$$

where $u^{n+p} = u(x_0 + ph)$

Generic FD

The truncated Taylor expansions for a given p can be written

$$u^{n+p} = \sum_{i=0}^M \frac{(ph)^i}{i!} u^{(i)}(x_0) = \sum_{i=0}^M c_{pi} du_i$$

using $c_{pi} = \frac{(ph)^i}{i!}$ and $du_i = u^{(i)}(x_0)$.

This can be understood as a matrix-vector product

$$\mathbf{u} = C \mathbf{du},$$

where $\mathbf{u} = (u^{n+p})_{p=p_0}^{M+p_0}$, $C = (c_{p_0+p,i})_{p,i=0}^{M,M}$ and $\mathbf{du} = (du_i)_{i=0}^M$. Here p_0 is an integer representing the lowest value of p in the stencil.

For $p_0 = -2$ and $M = 4$:

$$\mathbf{u} = (u^{n-2}, u^{n-1}, u^n, u^{n+1}, u^{n+2})^T \quad \mathbf{du} = (u^{(0)}, u^{(1)}, u^{(2)}, u^{(3)}, u^{(4)})^T$$

The stencil matrix

For $p_0 = -2$ and $M = 4$ we get 5 Taylor expansions

$$u^{n-2} = \sum_{i=0}^M \frac{(-2h)^i}{i!} du_i$$

$$u^{n-1} = \sum_{i=0}^M \frac{(-h)^i}{i!} du_i$$

$$u^n = du_0$$

$$u^{n+1} = \sum_{i=0}^M \frac{(h)^i}{i!} du_i$$

$$u^{n+2} = \sum_{i=0}^M \frac{(2h)^i}{i!} du_i$$

The stencil matrix ctd

Expanding the sums these 5 Taylor expansions can be written in matrix form

$$\underbrace{\begin{bmatrix} u^{n-2} \\ u^{n-1} \\ u^n \\ u^{n+1} \\ u^{n+2} \end{bmatrix}}_u = \underbrace{\begin{bmatrix} \frac{(-2h)^0}{0!} & \frac{(-2h)^1}{1!} & \frac{(-2h)^2}{2!} & \frac{(-2h)^3}{3!} & \frac{(-2h)^4}{4!} \\ \frac{(-h)^0}{0!} & \frac{(-h)^1}{1!} & \frac{(-h)^2}{2!} & \frac{(-h)^3}{3!} & \frac{(-h)^4}{4!} \\ 1 & 0 & 0 & 0 & 0 \\ \frac{(h)^0}{0!} & \frac{(h)^1}{1!} & \frac{(h)^2}{2!} & \frac{(h)^3}{3!} & \frac{(h)^4}{4!} \\ \frac{(2h)^0}{0!} & \frac{(2h)^1}{1!} & \frac{(2h)^2}{2!} & \frac{(2h)^3}{3!} & \frac{(2h)^4}{4!} \end{bmatrix}}_C \underbrace{\begin{bmatrix} du_0 \\ du_1 \\ du_2 \\ du_3 \\ du_4 \end{bmatrix}}_{du}$$

$$u = Cdu$$

Invert to obtain

$$du = C^{-1}u$$

Since du_i is an approximation to the i 'th derivative we can now compute any derivative stencil!

Second order 2nd derivative stencil

We have been using the following stencil

$$du_2 = u^{(2)}(x_0) = \frac{u^{n+1} - 2u^n + u^{n-1}}{h^2}.$$

Let's derive this with the approach above. The scheme is central and second order so we use $p_0 = -1$ and $M = 2$ (hence $p = (-1, 0, 1)$). Insert into the recipe for C

$$C = \begin{bmatrix} 1 & -h & \frac{h^2}{2} \\ 1 & 0 & 0 \\ 1 & h & \frac{h^2}{2} \end{bmatrix}$$

In Sympy

```
1 import sympy as sp
2 x, h = sp.symbols('x,h')
3 C = sp.Matrix([[1, -h, h**2/2], [1, 0, 0], [1, h, h**2/2]])
```

Print C matrix

```
1 C
```

$$\begin{bmatrix} 1 & -h & \frac{h^2}{2} \\ 1 & 0 & 0 \\ 1 & h & \frac{h^2}{2} \end{bmatrix}$$

Print C^{-1} matrix

```
1 C.inv()
```

$$\begin{bmatrix} 0 & 1 & 0 \\ -\frac{1}{2h} & 0 & \frac{1}{2h} \\ \frac{1}{h^2} & -\frac{2}{h^2} & \frac{1}{h^2} \end{bmatrix}$$

The last row in C^{-1} represents u'' ! The middle row represents a second order central u' .

Create a vector for u and print u' and u''

```
1 u = sp.Function('u')
2 coef = sp.Matrix([u(x-h), u(x), u(x+h)])
```

```
1 (C.inv()[1, :] @ coef)[0]
```

$$-\frac{u(-h+x)}{2h} + \frac{u(h+x)}{2h}$$

```
1 (C.inv()[2, :] @ coef)[0]
```

$$-\frac{2u(x)}{h^2} + \frac{u(-h+x)}{h^2} + \frac{u(h+x)}{h^2}$$

We can get any stencil

Create a function that computes C for any p_0 and M

```
1 def Cmat(p0, M):
2     C = np.zeros((M+1, M+1), dtype=object)
3     for j, p in enumerate(range(p0, p0+M+1)):
4         for i in range(M+1):
5             C[j, i] = (p*h)**i / sp.factorial(i)
6     return sp.Matrix(C)
```

$$p_0 = -1, M = 2$$

```
1 Cmat(-1, 2)
```

$$\begin{bmatrix} 1 & -h & \frac{h^2}{2} \\ 1 & 0 & 0 \\ 1 & h & \frac{h^2}{2} \end{bmatrix}$$

A central stencil of order l for the k 'th derivative requires $M + 1$ points, where

$$M = l + 2 \left\lfloor \frac{k-1}{2} \right\rfloor$$

Forward and backward

Non-central schemes requires one more point for the same accuracy as central

Forward u''

$$p_0 = 0, M = 3$$

```
1 C = Cmat(0, 3)
2 C.inv()
```

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ -\frac{11}{6h} & \frac{3}{h} & -\frac{3}{2h} & \frac{1}{3h} \\ \frac{2}{h^2} & -\frac{5}{h^2} & \frac{4}{h^2} & -\frac{1}{h^2} \\ -\frac{1}{h^3} & \frac{3}{h^3} & -\frac{3}{h^3} & \frac{1}{h^3} \end{bmatrix}$$

Backward u''

$$p_0 = -3, M = 3$$

```
1 C = Cmat(-3, 3)
2 C.inv()
```

$$\begin{bmatrix} 0 & 0 & 0 & 1 \\ -\frac{1}{3h} & \frac{3}{2h} & -\frac{3}{h} & \frac{11}{6h} \\ -\frac{1}{h^2} & \frac{4}{h^2} & -\frac{5}{h^2} & \frac{2}{h^2} \\ -\frac{1}{h^3} & \frac{3}{h^3} & -\frac{3}{h^3} & \frac{1}{h^3} \end{bmatrix}$$

Recognize the third row in the forward scheme:

$$u'' = \frac{-u^{n+3} + 4u^{n+2} - 5u^{n+1} + 2u^n}{h^2}$$

Fourth order central $p_0 = -2, M = 4$

```
1 C = Cmat(-2, 4)
2 C.inv()
```

$$\begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ \frac{1}{12h} & -\frac{2}{3h} & 0 & \frac{2}{3h} & -\frac{1}{12h} \\ -\frac{1}{12h^2} & \frac{4}{3h^2} & -\frac{5}{2h^2} & \frac{4}{3h^2} & -\frac{1}{12h^2} \\ -\frac{1}{2h^3} & \frac{1}{h^3} & 0 & -\frac{1}{h^3} & \frac{1}{2h^3} \\ \frac{1}{h^4} & -\frac{4}{h^4} & \frac{6}{h^4} & -\frac{4}{h^4} & \frac{1}{h^4} \end{bmatrix}$$

Third row gives us:

$$u'' = \frac{-u^{n+2} + 16u^{n+1} - 30u^n + 16u^{n-1} - u^{n-2}}{12h^2} + \mathcal{O}(h^4)$$

where the order $l = M - 2 \left\lfloor \frac{2-1}{2} \right\rfloor = M = 4$.

What is the order of the fourth derivative $u'''' = \frac{u^{n+2} - 4u^{n+1} + 6u^n - 4u^{n-1} + u^{n-2}}{h^4}$?

$$l = M - 2 \left\lfloor \frac{4-1}{2} \right\rfloor = M - 2 = 2$$